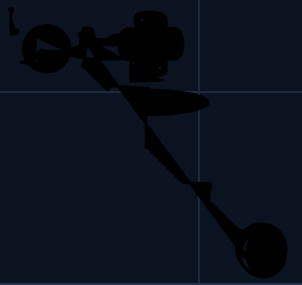
~/dev — uv

一个二进制，收干净Python装包那一摊

极快的 Python 包与项目管理器，用 Rust 写成

An extremely fast Python package and project manager, written in Rust.

uv 0.12.10 · released 2026-09-04 · docs.astral.sh/uv



pip　pip-tools　pipx　pyenv　virtualenv　twine

来源：uv 官方文档首页 (docs.astral.sh/uv)· 版本与发布日期取自 uv CHANGELOG

开场张力

你每天在修的，不是包，是工具之间的缝

pip	装包
pip-tools	锁版本
pipx	跑命令行工具
pyenv	管 Python 版本
virtualenv	建虚拟环境
twine	发包到索引

缝在哪

装 Python 与装包是两套工具，各有各的失败方式

requirements.txt 不跨平台，换台机器就重解一次

环境重建慢，于是没人愿意删掉重来

> Python tooling can be a low-confidence experience: it's a significant amount of work to stand up a new or existing project, and commands fail in confusing ways.
> — Astral,uv: Python packaging in Rust,2024-02-15

工具职责划分依据 uv 官方文档 Features 页；引文出自 Astral 发布博文 astral.sh/blog/uv

定位主张

一个入口，替掉一整排命令

one binary

uv

一个命令替掉：

pip pip-tools pipx poetry
pyenv twine virtualenv

A single tool to replace pip, pip-tools, pipx, poetry, pyenv, twine, virtualenv, and more.

出处：首页 Highlights 第一条

- 用 Rust 写成的单个静态二进制
- 不需要先有 Rust 或 Python 就能装
- 提供跨平台的 uv.lock，一份锁文件走遍机器

后面每一个倍数都会标出：
谁测的、什么机器、什么时候

能力揭示 · 项目工作流

从零到能跑，你只需要记住四个动作

`uv init` 建项目 → `uv add` 加依赖 → `uv run` 跑命令 → `uv lock` 锁版本

bash

```
$ uv init hello-world
$ cd hello-world
$ uv add requests
$ uv run hello-world
Hello from hello-world!
```

首次运行项目命令时，uv 自动创建 .venv 与 uv.lock

这四步留下什么

pyproject.toml
声明依赖与项目元数据

uv.lock
精确解析结果，应提交到版本库

.venv/
项目自己的隔离环境

命令与终端输出引自 uv 官方文档「Working on projects」页 (docs.astral.sh/uv/guides/projects)

演示时刻

看它跑一遍：五条命令，一次装完

bash — uv 0.12.10　　docs.astral.sh/uv

```
$ uv init example
Initialized project `example` at `/home/user/example`
$ cd example
$ uv add ruff
Creating virtualenvironment at: .venv
Resolved 2 packages in 170ms
Built example @ file:///home/user/example
Prepared 2 packages in 627ms
Installed 2 packages in  1ms
+ example==0.1.0 (from file:///home/user/example)
+ ruff==0.5.4
$ uv run ruff check
All checks passed!
$ uv lock
Resolved 2 packages in 0.33ms
```

以上为 uv 官方文档首页 Projects 段的示例会话，逐行照抄；机器与网络条件未公布，本页不做本机复测

来源：docs.astral.sh/uv 首页 Projects 段；高亮的耗时数字为该示例输出原文

一个脚本也能有自己的依赖，不用先建环境

bash — example.py

```
$ uv add --script example.py requests
Updated `example.py`
$ uv run example.py
Reading inline script metadata from: example.py
Installed 5 packages in 12ms
<Response [200]>
```

临时跑一次

```
uvx pycowsay 'hello world!'
```

uvx 即 uv tool run

装成常驻

```
uv tool install ruff
Installed 1 executable
```

此后直接敲 ruff 即可

内联依赖元数据

依赖写进脚本文件本身，不再靠全局环境

隔离环境

uv run 在隔离的虚拟环境里执行脚本

临时与常驻

uvx 跑一次；uv tool install 装成常驻

这两块与项目、 pip 接口一样，
都可以单独采用，不必整套迁移

命令与输出引自 uv 官方文档首页 Scripts / Tools 段与 Features 页 (docs.astral.sh/uv)

能力揭示 · Python 版本

连 Python 版本也归它管，不用再装一个 pyenv

bash — uv python

```
$ uv python install 3.10 3.11 3.12
Installed 3 versions in 3.42s
+ cpython-3.10.14-macos-aarch64-none
+ cpython-3.11.9-macos-aarch64-none
+ cpython-3.12.4-macos-aarch64-none
$ uv venv --python 3.12.0
Using CPython 3.12.0
Creating virtualenvironment at: .venv
$ uv python pin 3.11
Pinned `.python-version` to `3.11`
```

装多个 Python 版本

uv python install

需要时按需下载

uv venv --python 3.12.0

钉住当前目录

uv python pin 3.11

.python-version 决定 uv 为这个项目创建虚拟环境时使用哪个 Python 版本。
上面输出末尾的 macos-aarch64 说明这段官方示例是在 macOS 上执行的；你机器上的标识会不同。

命令与输出引自 uv 官方文档首页 Python versions 段与 Working on projects 页

前后对照

同一件事，你只是换了一条命令

任务	传统命令	uv 命令
建虚拟环境		
装包		
锁定依赖		
同步环境		
跑命令行工具		
装 Python 版本		

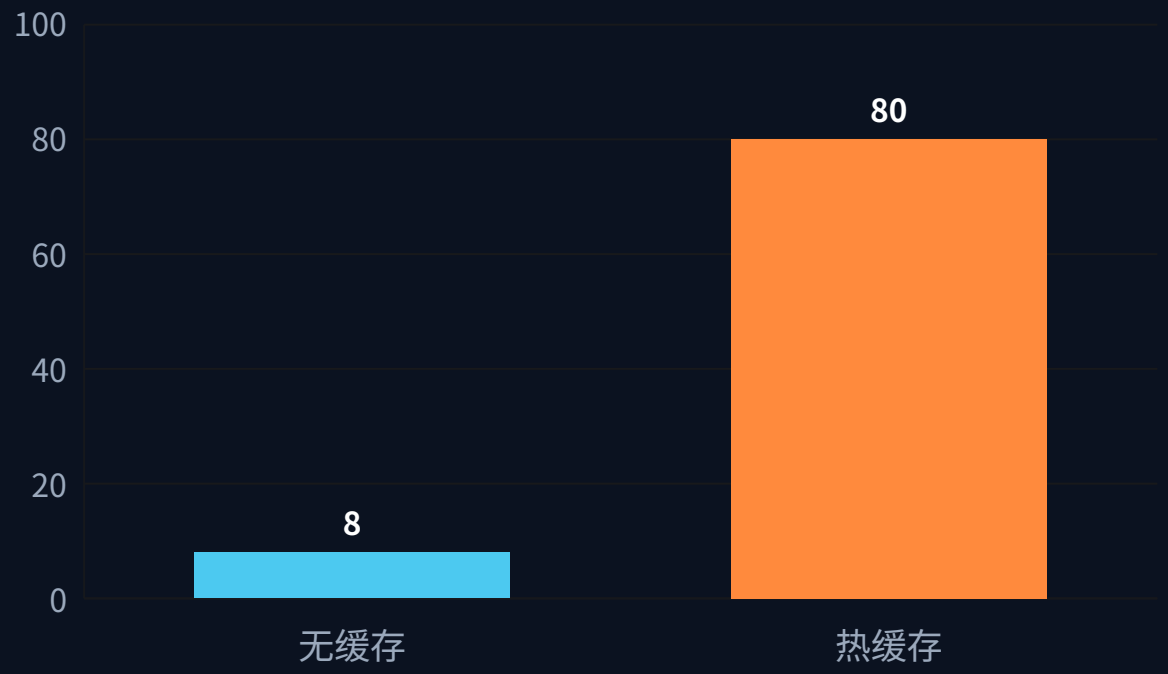
uv 不依赖也不调用 pip: 这个名字只表示这组低层命令与 pip 的接口对齐。
官方同时提醒：这些命令并非逐字实现 pip 的全部行为，越偏离常见工作流，差异越可能出现。

对照与两条提醒引自 uv 官方文档 The pip interface 页与 Features 页 (docs.astral.sh/uv/pip)

证据

快多少，取决于缓存是冷是热

单位：倍（相对 pip，越高越快）— 取 Astral 公布区间的下限



测量口径

测量者：Astral（uv 的作者）

平台：macOS，非 uv 工具用 Python 3.12.4

对象：Trio 的 docs-requirements.in

日期：2024-02-15（Astral 发布博文）

博文原句给的是区间，不是单一数字

cold 8-10x faster than pip
warm 80-115x faster than pip

不是你机器上的结果，本 deck 未做本机复测

官方文档首页另写作「10-100x faster than pip」，指向同一份 BENCHMARKS.md。
BENCHMARKS.md 提醒：不同操作系统与文件系统上差异可能很大，不同依赖集也会显著改变结果。

倍数出自 Astral 发布博文 astral.sh/blog/uv(2024-02-15)；测量条件出自 uv 仓库 BENCHMARKS.md

能力全景

你可以只用其中一块，也可以整套换过来

Python 版本	`uv python install/list/find/pin/uninstall`
脚本	`uv run/uv add --script/uv remove --script`
项目	`uv init/add/remove/sync/lock/run/tree/build/publish`
工具	`uvx (= uv tool run)/uv tool install/uninstall/list`
pip 接口	`uv venv/uv pip install/compile/sync/freeze/tree`
Utility	`uv cache clean/prune/dir · uv self update`

官方原话：uv 的界面可以拆成若干部分，这些部分既可以单独使用，也可以合起来用。

命令清单与分区引自 uv 官方文档 Features 页 (docs.astral.sh/uv/getting-started/features)

边界与状态

分清哪些能拿来用，哪些要显式开启

能力标识	它做什么	状态	见于版本
	uv init 默认声明 build system	已发布	
	拒绝清空非虚拟环境目录	已发布	
	uv run 从脚本所在目录发现项目	已发布	
	按内容哈希对缓存中的 wheel 去重	preview	
	按名字选已配置的索引	preview	
	报告实际回收的磁盘空间	preview	
	锁文件省略无关的新鲜度设置	preview	

```
$ uv add --preview-features index-by-name torch --index pytorch
```

版本取自 uv CHANGELOG: 已发布行是它稳定的版本，preview 行是它首次出现的版本；preview 不带稳定承诺，须显式开启

可获得性

三个平台都能装，而且不用先有 Python

 macOS  Linux  Windows 安装 uv 不需要先有 Rust 或 Python

macOS / Linux

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows PowerShell

```
PS> powershell -ExecutionPolicy ByPass -c
"irm https://astral.sh/uv/install.ps1 | iex"
```

其他渠道：pip 、 pipx 、 Homebrew 等，见官方安装页

已有项目升到 0.12：若 [build-system] 给 uv_build 设了上界，改成 uv_build>=0.11.32,<0.13
uv build backend 的配置本身没有破坏性变更，只需放开这一个版本上界

平台与安装方式引自 uv 官方文档首页 Installation 段；升级提醒引自 uv CHANGELOG 0.12.0(2026-07-28)

出处

本卷每一句的出处都在这四个地方

官方文档

命令、示例输出与能力划分 — 支撑 P03–P08 、 P10 、 P12

更新日志

版本号、发布日期、 preview 与稳定的分界 — 支撑 P01 、 P11 、 P12

发布博文

定位说法与 8-10x / 80-115x 的原始出处 (2024-02-15)— 支撑 P02 、 P09

基准说明

测量条件与告诫 , 本身不含数字 — 支撑 P09

本卷所有性能倍数均为 Astral 在 macOS 上的测量 , 未在本机复测 ; 命令与输出照官方文档原样引用。

四个链接均为原生超链接 , 在 PowerPoint 中可直接点击

下一步

今天先做一件事：在一个项目上跑通 uv pip install

first run

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
$ cd my-project
$ uv venv
$ uv pip install -r requirements.txt
```

先换一条命令，再谈换一套流程

uv 的虚拟环境符合标准，可与其他工具互换使用；uv pip 不调用 pip，越偏离常见工作流差异越可能出现

不必整套迁移：先在一个现有项目上跑通这两条命令，再决定要不要用 uv init 重建工程

安装命令与 pip 接口说明引自 uv 官方文档首页 Installation / The pip interface 段